

Flying Toasters Screen Saver

Note: Must commit to the **main** branch in your provided GitHub repository by assignment deadline
Please read the entire document before you begin.

Introduction

A long long time ago, when computer screens were based on cathode-ray tubes (CRTs), if the same image is displayed on screen for a long time, the phosphor coating inside the screen could easily be permanently damaged, leading to a screen burn-in, or darkened shadow on the screen. Screen-saver avoids this problem by automatically changing the images on the screen after the computer has been idle for a long time. Although modern computers do not have a screen burn-in issue, we still use screensavers for decorative purposes.

One of the most iconic examples was the “Flying Toasters” screensaver, which debuted in 1989 as part of the After Dark suite. In this MP, you will recreate a version of this classic screensaver using **RISC-V** assembly code.

MP1 Assignment

You will add 9 new functions to the existing code base.

Your code will reside in `mp1.s`, a **RISC-V** assembly file. Your code must be implemented using **RISC-V** assembly.

Frame Buffer

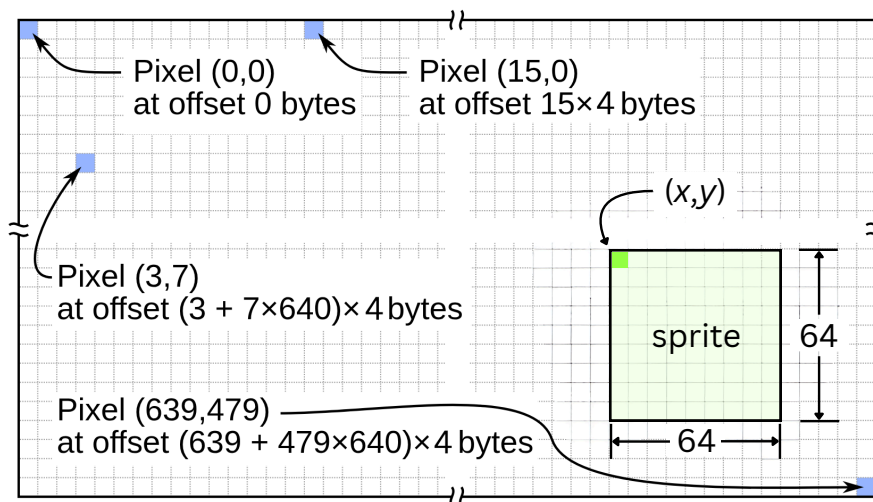


Figure 1: MP1 Screen

The frame buffer contains a representation of an image on the screen, where each pixel occupies fixed number of bytes. Each pixel is identified by its Cartesian coordinates (x, y) . The origin of the coordinate system is the upper left corner of the image. The x coordinate increases from left to right and the y coordinate from top to bottom (Note that the y coordinate orientation is opposite that of the usual Cartesian coordinate system in mathematics). The pixels are arranged in memory in row-major order, meaning that the pixels of the first row appear in memory before the pixels of the second, and the left-most pixel of the second row comes immediately after the right-most pixel of the first row.

In this assignment, we will use a frame buffer that is 640 pixels wide and 480 pixels high. (That is, there are 640 columns and 480 rows.) Each pixel contains the color data in RGB format and occupies 4 bytes of memory. Given this information, we can calculate the memory offset of each pixel from the starting address of the frame buffer as:

$$\text{memory offset} = (x + y \times 640) \times 4 \text{ bytes}$$

where we multiply by four because each pixel is four bytes in size.

Each pixel is 32 bits long, or 4 bytes, where the most significant byte defines the red value of the pixel, the second most significant byte represents the green value of the pixel, and the third most significant byte represents the blue value of the pixel. The least significant byte is a don't-care byte for the frame buffer pixel in memory - use it however you want. We represent the frame buffer by a `uint32_t` array called `fb_rgbx32`, whose dimensions are `FRAME_WIDTH × FRAME_HEIGHT`. In C, each of the colors of a pixel at (x, y) would be accessed as:

```
uint8_t red    = (uint8_t) (fb_rgbx32[x + y*FRAME_WIDTH] >> 24);
uint8_t green  = (uint8_t) (fb_rgbx32[x + y*FRAME_WIDTH] >> 16);
uint8_t blue   = (uint8_t) (fb_rgbx32[x + y*FRAME_WIDTH] >> 8);
```

Remember to consider the endianness – the order in which the most significant to least significant bytes of a `uint32_t` pixel are stored in memory.

MP1 Data Structure

```
struct toaster {
    uint8_t phase;      /* Which of the 4 animation frames of the toaster to display */
    int16_t x;          /* x-coordinate of the top-left pixel of the toaster */
    int16_t y;          /* y-coordinate of the top-left pixel of the toaster */
    char reserved[10]; /* Unused bytes */
}
```

A toaster is an animated 64×64 image, consisting of 4 animation frames. Each animation frame is a 64×64 image stored in row-major order. Each pixel consists of 3 bytes, in the order red, green, and blue. The `x` and `y` fields specify the (x, y) coordinate of the top-left pixel of the toaster. The `reserved` field are free to use and free to ignore.

The `phase` is an integer from 0 to 3, specifying the current animation frame of the toaster. Each animation frame is stored consecutively in memory, beginning at `toaster.start`. This means that the starting address of a toaster's pixel data is given by

```
toaster_start + toaster.phase * 64 * 64 * 3
```

To keep track of the toasters on screen, we will use a contiguous array of toasters called `toasters` in which you will add/remove toasters from. The order of the array does not matter, but all toasters must be contiguous in the array. The number of toasters in the array is tracked by the `toaster.cnt` variable.

```
struct toast {
    int16_t x;          /* x-coordinate of the top-left pixel of the toast */
    int16_t y;          /* y-coordinate of the top-left pixel of the toast */
    struct toast* next; /* Pointer to the next toast in the linked list */
}
```

A toast is a 64×64 image of pixels are stored in row-major order at the label `toast.start`. Each pixel consists of 3 bytes, in the order red, green, and blue. The `x` and `y` fields specify the (x, y) coordinate of the top-left pixel of the toast.

To keep track of the toasts on screen, we will use a linked list whose head is called `toast_list` in which you will add/remove toast from. The `next` field is a pointer to the next toast in the linked list. The order of the linked list does not matter.

MP1 Functions

Please refer to `after_dark.h` for global variable and function definition.

```
void draw_sprite(const char* sprite_rgb24, uint32_t* fbuf_rgbx32, int16_t x, int16_t y)
```

The `draw_sprite` function draws the given sprite `sprite_rgb24` buffer onto the frame buffer `fbuf_rgbx32`. A sprite is a 64×64 image of pixels stored in row-major order, where each pixel consists of 3 bytes (`rgb24`), in the order red, green, and blue. Do not assume that the sprite exists either in the toaster array or the toast linked list (During testing the function may be called with a sprite pointer that exists outside of the array or linked list). If either of these pointers is null, return immediately.

Draw the sprite beginning at its upper-left corner set at the pixel coordinates (`x`, `y`) within the frame buffer. Iterate over the pixels in the sprite and set the respective pixel in the frame buffer to the corresponding color. Note that `x` and `y` are signed integers, so they can be negative. Regardless, you should still draw the sprite starting from these coordinates, but clip it accordingly so that only pixels within the bounds of the frame buffer are written to. In other words, you should skip pixels that will not be within the screen. If a pixel in the sprite buffer is magenta (`R=0xFF`, `G=0x00`, `B=0xFF`), then the pixel is meant to be transparent, and you should also not draw these pixels and skip them. Remember to transform the pixel color from the 24-bit RGB format (`rgb24`) of the sprite buffer to the 32-bit RGB format (`rgbx32`) of the frame buffer when writing the pixel color data into the frame buffer. You do not need to worry about sprites overlapping each other.

```
void draw_toaster(struct toaster* t, uint32_t* fbuf_rgbx32)
```

The `draw_toaster` function draws the given toaster `t` into the frame buffer `fbuf_rgbx32`. Do not assume the given toaster `t` is already in the toaster array. If either pointer is null, return immediately. You should call `draw_sprite` to draw. Remember to select the appropriate image of the toaster sprite to draw based on the phase.

```
void draw_toast(struct toast* t, uint32_t* fbuf_rgbx32)
```

The `draw_toast` function draws the given toast `t` into the frame buffer `fbuf_rgbx32`. Do not assume the toast pointer argument `t` refers to a toast already in the toast linked list. If either pointer is null, return immediately. You should call `draw_sprite` to draw.

```
void move_toaster(struct toaster* t)
```

The `move_toaster` function moves the toaster 1 pixel down and 1 pixel to the left. It should also transition to the next phase. Because there are only 4 animation frames for the toaster, the phase must always be an integer from 0 to 3. If the current phase is 3, the next phase should be 0. If the pointer is null, return immediately.

```
void move_toast(struct toast* t)
```

The `move_toast` function moves the toast 1 pixel down and 1 pixel to the left. If the pointer is null, return immediately.

```
void add_toaster(int16_t x, int16_t y, uint8_t phase)
```

The `add_toaster` function adds a new toaster struct into the toaster array with the given `x`, `y` coordinates, and `phase`, and increments `toaster_cnt` by 1. If there is no room for the new toaster in the array, the request should be ignored (toaster array remains unchanged). The ordering of the toasters does not matter. The given `phase` will always be an integer from 0 to 3.

```
void add_toast(int16_t x, int16_t y)
```

The `add_toast` function dynamically allocates a new toast struct and initializes it with the given `x` and `y`, and adds it to the `toast_list` linked list. Ordering of toasts in the `toast_list` does not matter. If the `malloc` fails, then the linked list should not change.

```
void remove_toaster(int16_t x, int16_t y)
```

The `remove_toaster` function removes the toaster with the specified `x` and `y` coordinates from the toasters array and decrements `toaster_cnt` by 1. After removal, all toasters remaining in the array must be contiguous. This function does not need to preserve the order of any of the other toasters in the array. You can assume that there is at most one

toaster with the specified x and y coordinates. If there is no toaster in the array with the specified x and y coordinates, this function does nothing.

```
void remove_toast(int16_t x, int16_t y)
```

The `remove_toast` removes the toast with the specified x and y coordinates from the toast list. This function does not need to preserve the order of any of the other toast in the list. You can assume that there is at most one toast with the specified x and y coordinates. If there is no toast in the list with the specified x and y coordinates, this function does nothing. You must call `free` to free the memory allocated for the toast.

Allocating and Freeing Memory

User-level C programs make use of the `malloc()` and `free()` C library functions to allocate memory needed for storing dynamic structures such as linked list elements. The MPI distribution contains two memory allocation functions in `memory.c/h` that behave similarly to `malloc()` and `free()`. Their prototypes are:

```
void * malloc(size_t size);
void free(void * ptr);
```

`malloc` takes a parameter specifying the number of bytes of memory to allocate. It returns a `void*`, called a “void pointer”, which is the memory address of the newly-allocated memory.

`free` takes a pointer to a block of memory that was dynamically allocated with `malloc()` and releases/frees that memory back to the system. It does not return anything.

How to call the given function?

```
addi a0,x0,SIZE # put the proper size argument into a0
call malloc      # once the call is done, a0 will have the pointer to the allocated block
```

[Please refer to the documentation about the RISC-V LP64 ABI (or lecture content) for information about calling conventions]

Coding Style and Design

In general, being able to write readable code is a skill that’s just as important as being able to write working code. People and industry teams have their own preferences and rules when it comes to coding style. In this class, we won’t nitpick over small things such as spaces, blank lines, or camel case, nor will we enforce any rigorous coding guidelines. However, we still do have a basic standard that we expect you to adhere to and will be enforced through grading. Our expectations are outlined below:

- Give meaningful and descriptive (but not too long) names to your variables, labels, constants, functions, and files. Be consistent in your naming conventions.
- Do **NOT** use magic numbers (any number that appears in your code without a comment or meaningful symbolic name). -1 , 0 , and 1 are usually OK when used in obvious ways.
- Keep programs and functions relatively short. Don’t write spaghetti code that jumps back and forth everywhere. Create helper functions instead and make it easy to follow the flow of the program. Note that helper functions must obey the C calling convention.
- Use comments to explain the interfaces to all functions or subroutines, lengthy segments of code, and any non-obvious line of code. However, do **NOT** overdo it. Too many comments are just as bad as too few. Use comments to explain *why*, not *what*.
- Check for null pointers. A function should return immediately if the pointer is pointing to an invalid address.

- All function **MUST** obey the C calling convention. The autograder will check whether the registers are properly saved according to the convention.

As shown in the MPI rubric on the course website, 5pts are attributed to "full function interface for all functions". As a clarification on what that means, a function interface is a comment above the function that includes (at a minimum):

- The C function signature
- The input arguments to the function (types, names, and a 1-line description)
- The outputs of the function (types, names, and a 1-line description) ("None" if it is a void function)
- A description of the function's behavior (1-3 sentences)
- Any side effects of the function (if applicable)

As an example:

```
# uint32_t add(uint32_t a, uint32_t b)
# Inputs: uint32_t a - First operand
#         uint32_t b - Second operand
# Outputs: uint32_t sum of a and b
# Description: Returns the sum of a and b as a uint32_t.
# Side Effects: None
```

The function interface that goes in your code does not have to exactly follow this format, but it must include the same information. You need to provide a full function interface on every function that you write/modify.

How to get things started

Testing

For testing your code, we have provided a demo. You can run this demo using our "golden" implementation of each function using:

```
make clean
make demo-gold.elf
make run-demo-gold
```

You can run this demo using your implementation of each function using:

```
make clean
make demo.elf
make run-demo
```

Please note that just because the demo looks correct, your code may still have bugs. We recommend writing your own test cases to test each function individually.

Handin

Please push your code to the `main` branch under the `mp1` folder in your GitHub repository before the deadline.

Only commit source files (`.c`, `.h`, `.s`, etc.) and not object files (`.o`) other than the files we have already provided.

Code should remain within the `mp1` directory that was distributed. Do not change the name of the `mp1.s` file.

The posted rubric will contain additional information about the grading criteria.